
MemMux: Runtime Verification and Honest Resource Attribution for Fleets of Parallel Coding Agents

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Developers increasingly run not one coding agent but a fleet of them side
2 by side on one workstation. The tools they reach for, terminal multiplexers
3 like `tmux` and a new generation of agent managers, were built to arrange
4 windows, not to govern memory. When ten agents each spawn language
5 servers, test runners, and browsers, no standard tool in that stack can
6 say how much memory belongs to which agent, confirm that a terminated
7 agent’s descendants are gone, notice a child that has escaped its agent, or
8 keep the machine off the swap cliff when an OOM kill would silently discard
9 uncommitted work. We treat these as runtime-verification problems. An
10 agent-hosting substrate should continuously emit observable signals that
11 an operator or auditor can check while agents run. We present MEMMux,
12 a local runtime that turns resource governance into a set of checkable sig-
13 nals (per-agent attribution, complete reclamation, escaped-process visibil-
14 ity, bounded footprint under overcommit, and monitoring overhead) and
15 a claims-disciplined benchmark that measures each one against `tmux`, a
16 purpose-built agent multiplexer, and a raw-process baseline on identical
17 workloads. Under a binding memory budget on a Linux host, MEMMux
18 keeps the fleet under budget (7.5 GiB) with zero swap by admitting a sub-
19 set and reclaiming under pressure. The ungoverned tools instead run every
20 agent, pin the machine at its RAM ceiling ($2\times$ over budget), and spill about
21 2 GiB into swap. MEMMux reclaims 100% of a terminated agent’s process
22 subtree where the raw baseline strands half of it (ten orphaned processes,
23 303 MiB), and it is the only system that surfaces an escaped child, detecting
24 all 10 of 10 injected escapes. The cost is real and we report it: the always-on
25 1 Hz attribution scan runs near 0.6% CPU for one agent but reaches 2.7%
26 at ten, above our 2% target. Running the same harness on real Claude
27 Code sessions shows that 100% attribution and low overhead carry over to
28 live, multi-process agent trees. We release the engine, the benchmark, and
29 a one-command reproducer.

30 1 Introduction

31 A year ago, running an autonomous coding agent meant babysitting a single session. Today
32 it is normal to fan a task out across several agents at once—one refactoring, one writing tests,
33 one chasing a flaky build—and to watch them in a grid of panes. The agents themselves have
34 improved quickly [4, 10, 12, 13]; the substrate they run on has not. Developers arrange these

35 fleets with terminal multiplexers such as `tmux` [8] or with purpose-built agent managers, and
36 both were designed to place windows on a screen, not to answer questions about memory.

37 Those questions matter because a coding agent is not a single process. It is a tree: the
38 model’s CLI, a shell, language servers, test and build workers, sometimes a headless browser.
39 A handful of such trees can exhaust a laptop’s RAM. When they do, the operating system’s
40 response—the Linux OOM killer, or swap thrashing—is blunt and invisible: a process is
41 killed mid-write, or the whole machine grinds, and the agent’s uncommitted edits are simply
42 gone. Nothing in the stack was accountable for that memory in the first place. `tmux` does
43 not know that pane 7’s language server belongs to the same logical agent as pane 3’s test
44 runner; it cannot tell you that a double-forked child has escaped supervision; and it has no
45 budget it is trying to keep.

46 We think the right lens for this is *runtime verification* [1]: rather than prove properties of
47 the substrate ahead of time, instrument it so the properties it claims can be *checked while it*
48 *runs*, and surface any violation rather than hide it. Cluster schedulers made an analogous
49 move a decade ago—Borg combined admission control, over-commitment, and process-level
50 isolation, and treated monitoring as a first-class output [11]. We bring that stance down to
51 a single developer machine and to the specific failure modes of agent fleets.

52 This reframes an infrastructure problem as a verification one, on an axis the agent-evaluation
53 literature largely leaves open. Most work on verifying agents asks whether an agent’s *outputs*
54 are trustworthy: did it resolve the issue, is the reward genuine, is the behaviour aligned. We
55 ask a different question, one that matters more once agents run in parallel and unattended.
56 Does the *substrate that hosts them* uphold operational guarantees an operator can check as
57 the fleet runs: accurate accounting, complete reclamation, and no silently lost work? These
58 are always-on signals over the live execution, not one-shot scores. A fleet whose host cannot
59 answer them is unverified, however capable each agent is.

60 This paper contributes three things. First (Section 4), **five verification signals for agent**
61 **fleets**: per-agent attribution, complete reclamation on teardown, escaped-process visibil-
62 ity, bounded footprint (with preserved work) under overcommit, and monitoring overhead.
63 Four are runtime properties; the fifth is the monitor’s own cost. Each has a stated thresh-
64 old. Second (Section 3), **MemMux**, a local runtime that observes the signals and enforces
65 the governance properties. Third (Section 4), **a claims-disciplined benchmark** that
66 drives `tmux`, an agent multiplexer, and a raw baseline through *identical* deterministic work-
67 loads, tags every process as agent or manager overhead, and never reports a number it
68 did not measure. A competitor that cannot be driven fairly is recorded as skipped, not
69 estimated. Across two Linux hosts, a binding-budget experiment separates MEMMUX from
70 the ungoverned tools by keeping a twelve-agent stress workload under budget and off swap
71 (Section 5); Section 6 states the limits: mostly synthetic workloads, fleets no larger than
72 twelve, and an overhead target not yet met.

73 2 Background and Related Work

74 **Language-model agents.** Interleaving reasoning with tool use [13] and adding self-
75 reflection [10] turned language models into agents that take long sequences of actions. Cod-
76 ing is the canonical proving ground: SWE-bench frames the task as resolving real GitHub
77 issues against a repository [4], and SWE-agent shows a well-designed agent–computer inter-
78 face matters as much as the model [12]. Multi-agent settings, first studied for open-ended
79 behavior [9], are now a practical reality on developer machines. This work is orthogonal to
80 the agents: we govern whatever they spawn.

81 **Terminal multiplexers.** `tmux` [8] is the default way developers run many long-lived ses-
82 sions at once. It is excellent at persistent panes and detach/reattach, and it is the baseline
83 any “run many agents” tool must beat. But a pane is just a process group under a server,
84 with no notion of a logical agent, a memory budget, or a process that has wandered out of
85 its subtree.

86 **Resource management.** Cluster managers solved fleet-scale governance long ago: Borg
87 with admission control and over-commitment [11], Mesos with two-level scheduling [3], and
88 Kubernetes as the open-source lineage [2]. The enforcement primitives they use—Linux con-
89 trol groups [5] and the OOM killer [6]—live in every modern kernel. MEMMUX borrows the
90 ideas (reserve, admit, reclaim before you thrash) but targets a single un-orchestrated lap-
91 top, and it deliberately avoids hard cgroup containment in favor of signal-based supervision.
92 This is an explicit trade-off: cgroups give hard, kernel-enforced limits but need privilege
93 and per-machine setup, whereas signal-based supervision runs unprivileged and portably
94 (macOS included) at the cost of best-effort rather than hard containment—a child can, for
95 an instant, escape before reconciliation catches it (H3).

96 **Memory accounting.** Summing resident set size (RSS) over a process tree double-counts
97 shared pages. The proportional set size (PSS) exposed through `/proc/<pid>/smaps_rollup`
98 divides each shared page by its sharers and is the fair per-process figure [7]; macOS offers
99 `phys_footprint` as its analogue. MEMMUX attributes by PSS (RSS fallback) so a task’s
100 number reflects memory it is responsible for.

101 **Runtime verification.** Rather than verify a system statically, runtime verification instru-
102 ments it and checks a specification against the execution trace, reacting to violations [1].
103 Our first four signals are such monitors: cheap, always-on checks with explicit thresholds
104 whose failures are events, not silence; the fifth quantifies their runtime cost.

105 3 The Design of MemMux

106 MEMMUX is a Rust workspace: a daemon (`memmuxd`) that owns state and does the governing,
107 and a terminal client that attaches over a Unix domain socket. A *task* is a durable, logical
108 agent; a *runtime* is one physical launch of a provider inside a pseudo-terminal. The design
109 is organized around making each signal checkable (Figure 1).

110 **Attribution.** Once per second the daemon snapshots the host process tree and, for each
111 running task’s provider root, sums the PSS of the root and all descendants. That per-task
112 figure feeds three consumers: the budget, the client’s task view, and the escaped-process
113 check (which uses the pids recorded under each root). Attribution is scoped to processes
114 MEMMUX launched, so unrelated host activity never pollutes a task’s number.

115 **Recursive reclamation.** When a task is terminated (by the operator or because its
116 provider exited), MEMMUX reaps the *entire* owned subtree. Order matters: it signals the
117 subtree while the root is still alive, because once the root dies its children reparent to `init`
118 and the tree that identifies them is lost. For a provider that already exited, MEMMUX
119 reaps from the pids it recorded while the task ran. The target is at least 99.5% of owned
120 descendants gone within ten seconds.

121 **Escaped-process reconciliation.** A child that double-forks and reparents to `init` has
122 escaped its agent. This is the classic way a leak hides. Each sample, MEMMUX reconciles
123 the pids seen under a task against the live tree; a once-owned pid now living outside every
124 task subtree is flagged *escaped* and surfaced as an event. Reconciliation is scoped to the
125 managed set, so the signal is a real “we lost track of a process we started,” not host noise.

126 **Admission and the pressure ladder.** The daemon holds a memory budget (physical
127 RAM minus reserves for the OS, interactive apps, and a swap-safety margin). Starting a
128 task reserves its predicted peak; if the reservation would exceed the budget the task stays
129 queued with a visible reason rather than over-subscribing the machine. When live usage
130 climbs anyway, a graduated *pressure ladder* responds before the machine thrashes: reclaim
131 escaped children, hibernate the lowest-priority idle task, recycle the largest provider, and,
132 at the top of the ladder, terminate the lowest-priority tree, but only after persisting a
133 checkpoint of the victim’s Git state. Preserving work always precedes destroying a process.
134 Every governance action writes an audit decision with its reason and evidence, so the system
135 is auditable, not just automatic.

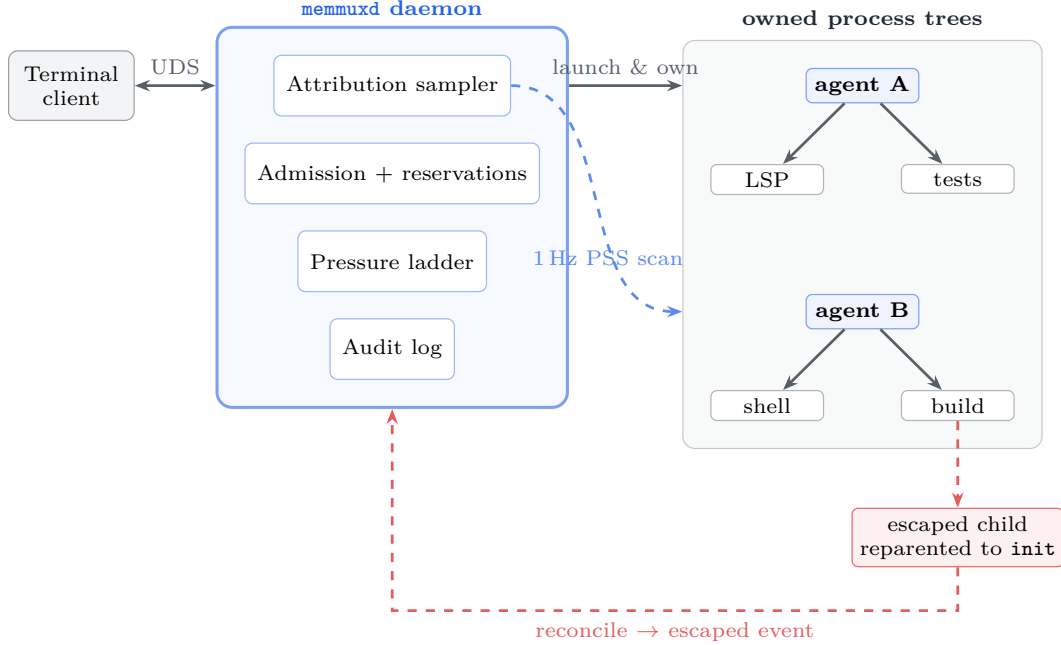


Figure 1: MEMMUX architecture. A terminal client drives the `memmuxd` daemon over a Unix domain socket. The daemon launches and *owns* each agent’s process tree; a 1 Hz sampler sums proportional memory (PSS) across every owned subtree (H1), reconciliation flags any child that double-forks out to `init` (H3), and admission, the pressure ladder, and an audit log turn the budget into checkable, logged decisions (H2/H4/H5).

136 4 Verification Signals and Methodology

137 We frame governance as runtime monitors over the fleet’s execution. The observed *trace*
138 is the sequence of once-per-second process-tree samples and lifecycle events; each signal
139 is an *invariant* over that trace with an explicit threshold (Table 1), and a breach is sur-
140 faced as a violation event rather than silently tolerated. Four are properties of the running
141 system (H1–H4); H5 is the *cost* of monitoring, not an invariant, and H4 bundles three sub-
142 properties—admission, bounded footprint, and preservation of uncommitted work—which
143 we report separately below. MEMMUX both *observes* these signals and *enforces* them (ad-
144 mission, reclamation); the baselines can only be observed. These are empirical monitors
145 with measured pass/fail behaviour, not machine-checked proofs, so we claim the signals are
146 checkable and checked, not that the engine is sound and complete in the formal sense. A
147 benchmark harness drives every launcher through identical, deterministic workloads and
148 evaluates the monitors from one external process sampler—an *oracle* using the daemon’s
149 own accounting code, pointed at whatever root pids a launcher reports, so measurement is
150 product-agnostic.

151 **Launchers.** Each launcher starts N identical agents and reports the pids that constitute
152 the fleet: a *raw* baseline (spawn the stub directly), `tmux` (one pane per agent on an isolated
153 server), and MEMMUX (create N tasks through the real daemon). A fourth, the `herdr` agent
154 multiplexer, is driven through its socket API where installed; on our stock Linux reference
155 host it was absent and is reported as skipped rather than guessed, as are `dmux` (which
156 requires a controlling terminal and cannot be driven headlessly), `cmux`, and `agentmux`.

157 **Workload.** A deterministic stub agent replays a recording of `allocate` / `spawn-child` /
158 `emit` / `sleep` steps, touching every page it allocates so its resident memory is real. We use a
159 stub rather than a live model on purpose: runs are then reproducible and identical across
160 launchers, at the cost of realism (Section 6). A *hold* scenario keeps N agents, each with

	Signal	Monitor / threshold
H1	Attribution	native signal maps $\geq 95\%$ of a launched tree’s proportional memory to its owning task
H2	Cleanup	$\geq 99.5\%$ of an owned subtree gone ≤ 10 s after the launcher’s normal teardown
H3	Escaped visibility	every previously observed process that escapes and remains live through the next sample is surfaced as an event
H4	Bounded / no-loss	footprint held near budget under overcommit; uncommitted work checkpointed before destructive termination
H5	Overhead	continuous monitoring costs $\leq 2\%$ CPU at fleet scale

Table 1: The five runtime-verification signals and their pass thresholds.

a live child, resident together for the measurement window. Without it, short workloads finish before a fleet can be observed, an error we hit early and corrected.

Claims discipline. The harness records each launcher’s version, the measurement date, and the host; it tags every sampled process as agent or manager overhead so the two are reported separately; and it emits a number only for a monitor it actually measured. A launcher that cannot be driven contributes “unsupported,” never a fabricated zero. These rules are part of the contribution: there is no accepted methodology for comparing agent-hosting tools, and an unfair comparison is worse than none. One point needs care. Because the oracle can attribute *any* launcher’s tree after the fact, per-agent attribution is *measurable* for all of them, and the real-agent run reports 100% for every launcher (Table 4). H1 as a signal asks the stricter question of whether a launcher exposes that attribution *natively and continuously* to an operator. Only MEMMUX does; `tmux` and the raw baseline surface no such figure themselves. Table 2 marks that native capability, not the oracle’s coverage.

5 Evaluation

Setup. Signals H1–H3 and H5 run on an AWS `m7i.2xlarge` reference host (8 vCPUs, Intel Xeon Platinum 8488C, ≈ 31 GiB RAM, Ubuntu 24.04), sweeping $N \in \{1, 5, 10\}$ over two trials. The bounded-footprint claim (H4) needs *real* memory pressure, so we run it on a deliberately smaller 15.3 GiB host (`m7i.xlarge`, 4 vCPUs, 12 GiB swap) with a calibrated synthetic stress footprint: each agent holds ~ 1.4 GiB, matching the Standard-class reservation prior, so admission is well-calibrated (reservation $\approx$ actual) rather than over-reserving, and $N=12$ agents (~ 18 GiB) genuinely overcommit the machine. The H1–H3/H5 sweeps use two trials as reproducibility checks and report their means without population-level confidence claims. The H4 run uses five trials; its archived report gives mean $\pm 95\%$ CI for footprint, while the table below rounds the means and reports observed peak swap. A macOS cross-check (Apple M4 Max, 32 GiB) that also exercised `herdr` agreed qualitatively: like the other ungoverned managers, `herdr` reaped its subtrees on teardown but saw no escapes and grew with the fleet. Table 2 summarises the head-to-head; the subsections that follow are the evidence behind each cell.

5.1 H4: bounded footprint under overcommit

We make the budget *binding*. Each agent in this calibrated synthetic stress workload holds ~ 1.4 GiB (the Standard-class reservation prior, so admission is well-calibrated rather than over-reserving a tiny stub), and $N=12$ such agents (~ 18 GiB) overcommit a 15.3 GiB host. MEMMUX is given a 7500 MiB budget—roughly half what the fleet wants. Table 3 reports peak total footprint and peak swap over five trials. The ungoverned launchers run all twelve agents: their resident set pins the machine at its ~ 15 GiB RAM ceiling— $2.0\times$ over the budget—and spills 1.9–2.2 GiB into swap. MEMMUX instead governs, holding peak footprint at 7479 MiB (*under* budget), settling to 4.5 GiB steady, with **zero** swap. Across

Launcher	Attrib.	Cleanup	Escaped	Footprint (budget)	Swap	Ckpt.
raw baseline	×	50%	×	15075 MiB	1882 MiB	×
tmux 3.4	×	100%	×	15347 MiB	2165 MiB	×
MEMMUX [‡]	✓	100%	✓ 10/10	7479 MiB	0	✓ [†]

Table 2: Head-to-head on the signals (Linux reference host). MEMMUX is the only launcher that natively attributes per-agent memory (H1), surfaces escaped processes (H3), holds the fleet *under* a binding memory budget with zero swap (H4), and can checkpoint dirty work before destructive termination. It *matches* **tmux** on teardown cleanup (H2). Footprint/swap are peaks under a binding 7500 MiB budget on a 15.3 GiB host ($N=12$); the ungoverned tools overcommit to the RAM ceiling and spill ~ 2 GiB into swap. Attribution is the *native* capability, not the oracle’s coverage (§4); MEMMUX also pays a monitoring cost the baselines do not (H5, §5). [†]Checkpoint-before-destructive-termination is confirmed by a daemon-level unit test, not triggered end-to-end. [‡]H1–H3/H5 use MEMMUX 0.8.0; H4 uses 0.9.0. Each cell is the headline result for that signal.

Launcher	Peak (MiB)	Steady (MiB)	Peak swap (MiB)
raw baseline	15075	15069	1882
tmux 3.4	15347	15069	2165
MEMMUX 0.9.0	7479	4512	0

Table 3: Binding-budget *overcommit* ($N=12$ agents, ~ 1.4 GiB each, 15.3 GiB host, five trials). MEMMUX’s 7500 MiB budget is half the ~ 15 GiB the ungoverned fleet uses; MEMMUX holds peak footprint under budget with zero swap, while raw and **tmux** overcommit the machine $2\times$ and spill ~ 2 GiB into swap. Worst-case maxima across trials were 7 deferrals and 12 reclamation events, not necessarily in the same trial.

198 trials, the respective worst-case maxima were seven deferred starts and twelve reclamation
199 events; these are event counts rather than unique tasks and can come from different trials.

200 The trade-off is throughput. MEMMUX admits at most about five agents and can settle lower
201 as pressure acts, so enforcing the budget means not running every agent at once. That is
202 what a budget is for: the ungoverned tools run all twelve only by driving the machine into
203 swap, where an OOM kill can strike. The *no-lost-work* half of H4 covers that case. Before
204 the pressure ladder’s *emergency* stage destructively terminates a victim, it first captures
205 a durable checkpoint (git HEAD + patch hash). In this run the reclamation events were
206 hibernate/recycle actions that never reached emergency termination, so this path was not
207 exercised end to end. A daemon-level test confirms it directly: a task with a dirty worktree,
208 forced into the emergency stage, produces a checkpoint carrying its patch hash *before* the
209 process is terminated. Ungoverned launchers have no checkpoint mechanism at all.

210 5.2 H2: complete cleanup on teardown

211 Each launcher starts a fleet whose agents each hold a memory-bearing child, then is torn
212 down exactly as it normally would be (raw: kill the roots; **tmux**: **kill-server**; MEMMUX:
213 terminate each task). Ten seconds later we count survivors from the owned set. The raw
214 baseline strands every child it spawned—its teardown kills only the roots, orphaning the
215 grandchildren—and the leak grows with the fleet: one process and 30.6 MiB at $N=1$, ten
216 processes and 303 MiB at $N=10$ (50% reclaimed at every size). **tmux** and MEMMUX both
217 reap their full subtree at every size (0 leaked, 100% reclaimed). On *explicit* teardown, then,
218 cleanup separates MEMMUX from the raw baseline but not from a real multiplexer: **tmux**
219 reaps its subtrees too. This is the kind of result the claims discipline is meant to surface.

220 5.3 H3: escaped-process visibility

221 We inject escapes: a stub forks an intermediate that spawns a memory-holding grandchild
222 and then exits, reparenting the grandchild to **init** while it stays alive. The harness inde-
223 pendently confirms the reparenting, then asks each launcher what it detected. MEMMUX

surfaces every escape—1/1, 5/5, 10/10—through events read from the daemon; the base-
lines have no mechanism to notice, so they are recorded as unsupported, never scored as
zero-of-measured. This is a capability rather than a tuning difference, and it is the clearest
separation from a real competitor. Detection is bounded by the sample interval. Reconcilia-
tion runs at 1 Hz, so a child that both escapes *and* exits within a second could slip between
two samples. Our injected escapes hold memory for several seconds, well past the interval;
a shorter interval would trade detection latency for overhead (H5). A persistent leak, which
is the case that matters, is caught on the next sample regardless.

232 5.4 H1 & H5: what the attribution scan delivers, and what it costs

We pair these two signals because they are the benefit and the price of a single mecha-
nism: the always-on 1 Hz PSS scan produces the attribution guarantee (H1) and incurs the
monitoring cost (H5). Across every run MEMMUX attributed 100% of each launched tree’s
proportional memory to its owning task *natively*, and the externally run oracle confirmed the
figure using the shared accounting implementation (H1). This validates the product integra-
tion but does not rule out a common-mode accounting bug; an independently implemented
oracle remains future work. The memory cost of governance is small: the MEMMUX dae-
mon’s own resident footprint is 4.6–5.4 MiB (growing slightly with the fleet), against 2.4 MiB
for the `tmux` server and zero for the raw baseline. The CPU cost is the weak spot (H5). The
MEMMUX daemon’s total CPU, dominated by the always-on 1 Hz full-tree PSS scan (we
did not ablate the two), is 0.6% at one agent, 1.5% at five, and 2.7% at ten, above our 2%
target once the fleet reaches ten. The scan is $O(\text{processes})$ and currently re-reads the whole
tree each tick; incremental sampling that touches only changed pids is the obvious fix and
is future work. We report the miss rather than hide it.

247 5.5 Validation on real coding agents

To test whether the attribution and overhead results are artifacts of the stub, we ran the
same harness against a real coding agent. Each “agent” is a headless Claude Code ses-
sion (`claude -p`, v2.1.170) editing a scratch Git repository, launched through the identical
sampling and teardown pipeline via the benchmark’s `--agent-cmd` mode. We report an
 $N=3$ hold run on the macOS host (Apple M4 Max, 36 GiB). A live Claude Code session
is a multi-process, Node-based tree, so this is a strictly harder attribution target than the
single-process stub.

Table 4 gives the result. MEMMUX attributes 100% of every real agent tree to its owning
task—the H1 gate holds on live agents, not just the stub—and the manager cost stays small
(7.2 MiB resident, 0.012% CPU at three agents, consistent with the small-fleet end of the H5
curve). Three concurrent real sessions occupy ~ 0.85 GB (~ 290 MiB each), confirming that
the pressure the system governs is real and per-agent-substantial. Attribution and overhead
thus generalize from the stub to real, multi-process agent trees.

Cleanup on live agents. That hold run let every session finish, so all launchers reclaimed
cleanly. To check the cleanup *mechanism* on *live* agents we ran a second experiment (Linux,
`m7i.xlarge`) in which three real Claude Code agents (v2.1.197) are *forcibly torn down mid-*
work, while their `node` process trees are still resident. The raw baseline strands **three real**
processes and 525 MiB of live agent memory (50% reclaimed)—killing the root `node`
orphans its children to `init`, exactly as with the stub—while both `tmux` and MEMMUX
reap the full subtree (0 leaked, 100%). Consistent with H2, this is a *raw*-versus-governed
result, not a separation from `tmux`; its value is that the cleanup behaviour the stub isolates
reproduces on real agent trees, and that the naive “run it in a terminal” baseline strands
half a gigabyte of live agent memory. The escaped-process condition (H3) did not arise here,
as Claude Code does not double-fork out of supervision.

272 6 Discussion and Limitations

Synthetic workload. The largest threat to validity: agents are a deterministic memory-
ballast stub, not live models. We trade realism for reproducibility and for identical workloads

Table 4: Real Claude Code agents ($N=3$, hold scenario, macOS) driven through the identical harness. Every launcher attributes the full real (Node-based) agent tree; MEMMUX’s manager overhead is small. Footprint is steady-state provider+manager memory; at $N=3$ without an overcommit budget the totals are comparable across launchers—this run isolates *attribution and overhead on real trees*, not the overcommit footprint differentiator of Table 3.

Launcher	Total steady (MiB)	Manager (MiB)	Tree attributed
raw baseline	883.6	0.0	100%
tmux 3.6a	883.6	2.4	100%
herdr 0.8.0	929.6	18.2	100%
MEMMUX 0.8.0	843.5	7.2	100%

across launchers, which is what makes the comparison fair. The *qualitative* guarantees—
attribution, cleanup, escape visibility, admission—do not depend on the workload, but the
specific megabytes for a real Claude- or GPT-driven session will differ. We take a first step
in §5.5, running real Claude Code sessions through the same harness: 100% attribution and
low overhead hold on live, multi-process agent trees, and forcibly killing them mid-work
reproduces the raw baseline’s cleanup leak (525 MiB) that the stub isolates. A larger live-
agent study—more agents and longer sessions, and reproducing the H4 overcommit and H3
escape conditions on live trees—remains the most valuable next step.

Two Linux hosts, modest fleets. We evaluate H1–H3/H5 on a 31 GiB Linux host, H4
on a 15.3 GiB Linux host, and perform a macOS cross-check. Fleets are no larger than
 $N=12$; larger fleets would make the CPU-overhead question (H5) sharper.

Static reservations. MEMMUX reserves each task’s *class-predicted* peak—a static prior.
Our H4 run sizes agents to that prior on purpose, so admission is well-calibrated; but
for an atypically small agent the prior over-reserves (deferring starts that would have fit),
and for an atypically large one it under-reserves. An exponentially-weighted predictor that
tightens the reservation toward each task’s *observed* peak is designed and is future work; the
bounded-footprint guarantee is unaffected—only the admit count moves with the predictor’s
accuracy.

Overhead not yet met. Monitoring exceeds 2% CPU by ten agents (above); incremental
sampling is the planned fix. As stated in §4, these are empirical monitors, not machine-
checked proofs—checkable and checked, not formally verified.

7 Reproducibility

MEMMUX and its benchmark are open source (MIT).¹ A single command runs the full
matrix—every launcher, scenario, and fleet size, with repeated trials—captures the host
specification, and regenerates every table and figure here plus the raw per-sample and per-
process time series. Competitor versions and the measurement date are embedded in the
generated report, and any launcher that could not be driven is listed with its reason. Re-
producing the evaluation on another host is one invocation. The same harness accepts a
`--agent-cmd` flag that substitutes a real coding-agent command (e.g. `claude -p ...`) for
the stub, so the real-agent validation of §5.5 is reproducible through the identical pipeline;
that run’s report and raw per-process series are archived under `paper/realagent-run/`.

Each run captures a host specification, a Markdown report, and per-cell time series (RSS,
PSS, USS, swap, and page faults per process on Linux) plus the figures. We withhold the
exact invocation and the repository here to preserve review anonymity; the engine, the
benchmark, and the one-command reproducer—including the binding-budget H4 configura-
tion used for Table 3—are released with the code.

¹Code, benchmark, and a one-command reproducer are open-source and will be released; the repository
URL is withheld to preserve double-blind review.

8 Conclusion

Fleets of coding agents have outrun the tools we use to run them: multiplexers arrange them, but nothing governs them. We argued that memory governance for such fleets is a verification problem and built MEMMUX to make five signals checkable at runtime, with a benchmark that also reports where those signals do *not* distinguish us. The payoff is concrete. Under a binding budget MEMMUX holds the fleet under budget and *off swap* where the ungoverned tools overcommit the machine ($2\times$ over budget, ~ 2 GiB into swap); it reclaims a terminated agent’s whole subtree where the raw baseline strands half a fleet; and it surfaces persistent escaped processes that the alternatives cannot see. Its own overhead we have not yet driven under 2%. The path from here is larger and longer live-agent workloads, including live H3/H4 stress conditions, and the reproducer makes those a single command away. Checking what an agent produced is only half of verification; the other half is checking that the system running it kept its promises, and the signals and claims discipline here are a step toward that.

References

- [1] Ezio Bartocci, Yliès Falcone, Adrian Francalanza, and Giles Reger. Introduction to runtime verification. In *Lectures on Runtime Verification: Introductory and Advanced Topics*, volume 10457 of *Lecture Notes in Computer Science*, pages 1–33. Springer, 2018. doi: 10.1007/978-3-319-75632-5_1.
- [2] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, Omega, and Kubernetes. *Communications of the ACM*, 59(5):50–57, 2016. doi: 10.1145/2890784.
- [3] Benjamin Hindman, Andy Konwinski, Matei Zaharia, Ali Ghodsi, Anthony D. Joseph, Randy Katz, Scott Shenker, and Ion Stoica. Mesos: A platform for fine-grained resource sharing in the data center. In *Proceedings of the 8th USENIX Conference on Networked Systems Design and Implementation (NSDI)*, 2011.
- [4] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.
- [5] Linux Kernel Documentation. Control group v2. <https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html>, 2024. Accessed 2026-08.
- [6] Linux Kernel Documentation. Out of memory management. <https://www.kernel.org/doc/html/latest/admin-guide/mm/concepts.html>, 2024. Accessed 2026-08.
- [7] Linux Kernel Documentation. The /proc filesystem: smaps and smaps_rollup (proportional set size). <https://www.kernel.org/doc/html/latest/filesystems/proc.html>, 2024. Accessed 2026-08.
- [8] Nicholas Marriott and tmux contributors. tmux: a terminal multiplexer. <https://github.com/tmux/tmux>, 2024. Accessed 2026-08.
- [9] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023. arXiv:2304.03442.
- [10] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.11366.
- [11] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at Google with Borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys)*, 2015. doi: 10.1145/2741948.2741964.
- [12] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.
- [13] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.